1. Fundamentals of cloud computing by uploading and retrieving files from AWS S3 using a Python script

Aim:

To demonstrate the fundamentals of cloud computing by uploading and retrieving files from AWS S3 using a Python script.

Procedure:

1. Set up an AWS S3 Bucket with necessary permissions.
2. Create an IAM User with S3 access and obtain access keys.
3. Use the AWS SDK (Boto3) in a Python script to upload and download files.
4. Verify file upload and retrieval from the S3 bucket.

PROGRAM:

```
import boto3
import os
# S3 Configuration
bucket_name = 'karthikeyan-81883177059'
local_file = 'bmi.csv'
s3_key = 'uploaded-kk.txt'
# Create S3 client (credentials loaded securely)
s3 = boto3.client( 's3', aws_access_key_id='AKIA35JRHIKRRR6PLWZV',
        aws_secret_access_key='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',
        region_name='us-east-1'
        )
def upload_file():
    s3.upload_file(local_file, bucket_name, s3_key)
    print(f"{local_file} uploaded to s3://{bucket_name}/{s3_key}")
def download_file():
    download_path = f"downloaded-{local_file}"
    s3.download_file(bucket_name, s3_key, download_path)
    print(f"File downloaded as {download_path}")
upload_file()
download_file()
```

Result:

Successfully demonstrated cloud storage using AWS S3, highlighting cloud computing's flexibility and scalability.

3. implimentation of contarinerization using docker for Python Application

Aim:

To create a Docker image for a Python application, run it as a container, and understand basic Docker commands.

Procedure:

1. Docker installation is verified to ensure the Docker service is running properly.
2. A project directory is created and required files such as the Python program and Dockerfile are prepared.
3. A Docker image is built using the Dockerfile to package the application and its dependencies.
4. The built Docker image is verified by listing available images.
5. A Docker container is run from the image to execute the Python application.
6. The application output is observed to confirm successful execution.

Output :

Hello from Flask running inside Docker!

Result:

The Docker image was successfully created and the Python application was executed using a Docker container.

.

5.Google App Engine using Render

Aim

To design and deploy a lightweight web application using the Flask framework that implements user authentication and session management. The project aims to provide a secure portal that displays dynamic cloud-related facts only to authorized users.

Procedure

1. **Initialize Application**: Setup the Flask environment and define a secret_key to track secure user sessions.
2. **Define Templates**: Create LOGIN_HTML and PORTAL_HTML strings with embedded CSS for a clean, centered interface.
3. **Handle Authentication**: Create a login route (/) that validates user input against pre-defined credentials (admin/123).
4. **Protect Portal**: Implement a /portal route that checks for an active session, redirecting unauthorized users to the login page.
5. **Serve Dynamic Content**: Use the random library to pick and display a different cloud fact upon every page refresh.
6. **Manage Sessions**: Develop a /logout route to clear session data and securely redirect the user back to the home screen.

Result

The Flask application was successfully developed and deployed, providing a functional login system and a dynamic portal. Authorized users can securely access cloud facts, while unauthorized access is prevented through session validation.

6. Creating a wharehouse Applications in sales Force.com

Aim

To design and implement a Warehouse Management System in Salesforce using Apex Classes and Triggers to manage stock, create orders, and validate stock availability before order insertion.

Procedure

1. Create Custom Object Product__c with fields Name and Stock_Quantity__c (Number).

2. Create Custom Object Order__c with fields Product_Name__c (Text) and Quantity__c (Number).

3. Create Apex Class WarehouseManagement with methods updateStock() and createOrder().

4. Create Trigger StockValidation on Order__c (before insert) to validate stock.

5. Use Execute Anonymous Window to insert sample records and test functionality.

Result

The Warehouse Management System was successfully implemented. Stock is updated correctly, orders are validated, and insufficient stock transactions are prevented using Apex class and trigger.

Left side

Output
1. Product records inserted successfully.
2. Query shows Laptop (50), Mobile (30), Tablet (20).
3. When attempting to create order for 100 Laptops,
   Debug Output: 'Insufficient stock'
4. Order insertion prevented when stock is insufficient.

9. implementation of para- virtualization using Virtual Box Creation and Execute Linux Commands

Aim

To create a Virtual Machine using VirtualBox and execute basic Linux terminal commands.

Procedure

1. Install VirtualBox on the system.

2. Download the Ubuntu ISO image file.

3. Open VirtualBox and click New to create a new virtual machine.

4. Enter the VM name and select:

Type: Linux

Version: Ubuntu (64-bit).

Allocate RAM (minimum 2048 MB).

Create a virtual hard disk:

Select VDI (VirtualBox Disk Image)

Choose Dynamically Allocated

Set disk size to 20 GB.

5. Go to Settings → Storage and attach the Ubuntu ISO file.

6. Start the virtual machine and install Ubuntu.

7. After installation, open the Terminal.

8. Execute basic Linux commands.

Result

The Virtual Machine was successfully created using VirtualBox and basic Linux commands were executed in the Ubuntu terminal.